

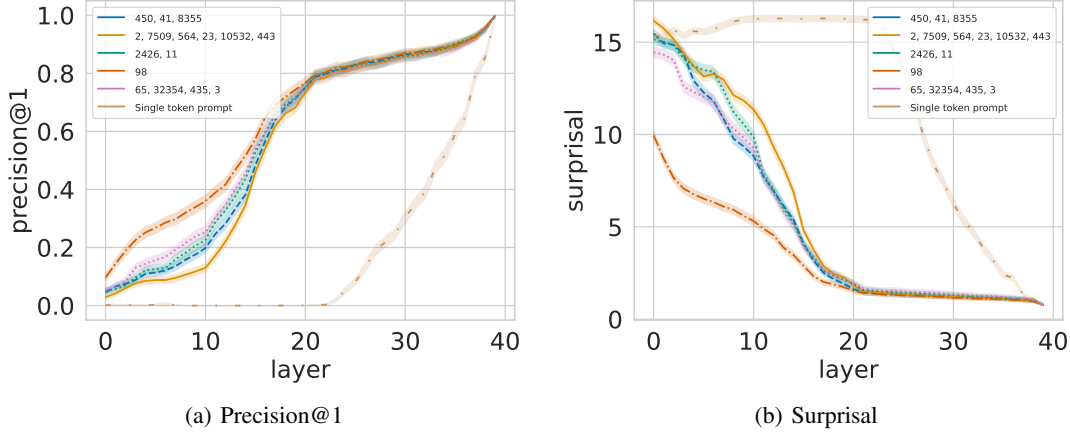


Figure 5. Next-token prediction results for LLaMA2, using various token identity demonstrations (the token IDs appear in the legend). We report precision@1 ($\uparrow$ is better), and surprisal ($\downarrow$ is better).

user-shared conversations⁸. The primary architectural differences between LLaMA2 and Vicuna (LLaMA1) include a different context length and grouped-query attention. Pythia and GPT-J were pre-trained using a deduplicated version of the Pile corpus⁹ (Gao et al., 2020), and for about 300B and 402B tokens, respectively.

B.2. Training and Evaluation Data

We use 12,000 random samples from the Pile, partitioned into 10,000 examples for training the affine mappings, and 2,000 examples for evaluation. In our pre-processing strategy, we introduce randomness in the patching positions by trimming the input sequence length of each example.

B.3. Additional Few-Shot Token Identity Prompts

In this section, we provide additional details about the selection of the demonstrations for the token identity baseline, and further evaluate the robustness of LLaMA2 (13B) (Touvron et al., 2023b) to various token identity prompts.

Demonstrations Construction For the demonstrations used in this experiment, we sample a random set of k tokens for all the models, where k was also randomly sampled from the interval $[1, \dots, 10]$.

Robustness to Additional token IDs’ Demonstrations We randomly generate five realizations of token IDs series of varying lengths, formatted as “tok₁ → tok₁ ; tok₂ → tok₂ ; ... ; tok_k”, similarly to the procedure from §4.1. We also include an ablation experiment, introducing the *Single Token Prompt* baseline. In this approach, the representation is transferred from the original multi-token prompt to a single-token prompt, while preserving the same position. This is crucial for maintaining consistent positional biases during the computation in the forward pass. The baseline is implemented by creating a target prompt in the format: “[PAD] [PAD] ... [PAD] tok_k”. Here, the number of [PAD] tokens is chosen such that “tok_k” retains its original position from the source prompt.

The results are illustrated in Fig. 5, where a comprehensive overview of the evaluation metrics can be found in §4.1. The results indicate the stability of the token identity baseline across a range of token identity demonstrations, particularly notable in the upper layers of the model. The *Single Token Prompt* baseline overall seems to be less effective than using our token ID baseline.

Table 5. Comparison between the zero-shot feature extraction Patchscope and the logistic regression probe shows that despite using *no training data*, the Patchscope has a significantly higher accuracy than the baseline in most tasks ($p < 1e - 5$). Pairwise t-statistics and the corresponding p-values are included in the table. The columns corresponding to each method show accuracy (mean $\pm$ std).

	Task	Probe	Patchscope	T-statistic	p-value
Commonsense	Fruit inside color	37.41 $\pm$ 6.58	37.99 $\pm$ 18.67	0.126	0.901
	Fruit outside color	35.50 $\pm$ 3.09	71.00 $\pm$ 13.26**	12.426	$< 1e - 5$
	Object superclass	68.92 $\pm$ 10.69**	55.71 $\pm$ 10.81	-5.25	$< 1e - 4$
	Substance phase	73.77 $\pm$ 3.74	91.92 $\pm$ 1.73**	25.647	$< 1e - 5$
	Task done by person	0 $\pm$ 0	62.96 $\pm$ 16.513**	19.632	$< 1e - 5$
	Task done by tool	10.14 $\pm$ 3.23	48.12 $\pm$ 13.23**	18.231	$< 1e - 5$
	Work location	0 $\pm$ 0	13.58 $\pm$ 9.37**	7.45990	$< 1e - 5$
Factual	Company CEO	4.99 $\pm$ 2.56	47.82 $\pm$ 13.89**	16.700	$< 1e - 5$
	Country capital city	0 $\pm$ 0	61.61 $\pm$ 14.14**	22.426	$< 1e - 5$
	Country currency	17.70 $\pm$ 2.20	50.95 $\pm$ 8.85**	20.293	$< 1e - 5$
	Country largest city	0 $\pm$ 0	67.78 $\pm$ 11.47**	30.427	$< 1e - 5$
	Food from country	5.13 $\pm$ 3.66	63.80 $\pm$ 11.34**	26.710	$< 1e - 5$
	Person father	0 $\pm$ 0	25.34 $\pm$ 8.42**	15.482	$< 1e - 5$
	Person plays position in sport	75.89 $\pm$ 9.14	72.20 $\pm$ 7.21	-2.066	0.049
	Person plays pro sport	53.87 $\pm$ 10.28	46.28 $\pm$ 14.19	-2.020	0.054
	Product by company	58.91 $\pm$ 7.15	63.24 $\pm$ 10.74	1.757	0.091
	Star constellation	17.54 $\pm$ 5.30	18.35 $\pm$ 5.06	-2.98	0.006
	Superhero archnemesis	0 $\pm$ 0	41.73 $\pm$ 18.72**	11.47044	$< 1e - 5$
	Superhero person	0 $\pm$ 0	28.32 $\pm$ 14.05**	10.37461	$< 1e - 5$

C. More Details on Attribute Extraction Experiments

Dataset Details We start from the factual and commonsense reasoning subsets introduced by Hernandez et al. (2023b)¹⁰. This dataset includes 8 commonsense and 25 factual relations. Recall that each datapoint is representing a triplet (σ, ρ, ω) , where σ refers to the subject, ρ represents the relation, and ω stands for the object (see 4.2 for notation details). For each data point representing the triplet (σ, ρ, ω) , we sample 5 utterances from Wikitext-103 dataset (Merity et al., 2016) including σ . We then truncate the sampled text to a window of random length up to 20 tokens that contains σ . This constitutes our source prompt, S . Note that for each model, we filter the data to samples for which the underlying model correctly encodes the tuple. For experiments with zero-shot target prompt T that includes ρ followed by σ , we autoregressively generate the next 20 tokens, and only keep examples where ω appears in the generation. For experiments with few-shot demonstrations in T , after generating the next 20 tokens, we do an additional post-processing step. If the demonstration template of an example is identified in the generation, all the following tokens would be dropped. The example is used for evaluation only if ω appears in the post-processed truncated generation. To have a reasonable amount of data for training the classification probe baseline, tasks with fewer than 15 datapoints are dropped from the analysis. For GPT-J, 5 commonsense and 7 factual reasoning tasks remain after applying the above postprocessing steps.

Results on Additional Tasks We provide additional results on various factual and commonsense reasoning tasks. For a comparison of the zero-shot feature extraction Patchscope and the logistic regression probe averaged across layers, see t-statistic details in Tab. 5.

Performance Breakdown Across Source Layers Fig. 6 depicts how accuracy changes across source layers $\ell \in [1, \dots, L]$. Considering the early layers, we observe that the Patchscope consistently outperforms the baseline. This further confirms our hypothesis that prior methods are particularly limited in surfacing information in the early layers, which often cannot be decoded via linear functions. However, Patchscope is able to extract such attributes significantly earlier. Note that this observation does not mean the attribute is *explicitly* encoded in a representation, but that there is enough information encoded such that the attribute can be extracted from the representation alone, without its original context, using the model’s computation. In the middle layers, Patchscope works similarly or better than the baseline. Interestingly, we observe

⁸Conversations were collected from www.sharegpt.com.

⁹Pile dataset is available under MIT license.

¹⁰The dataset released by Hernandez et al. (2023b) is available under MIT license.

that almost all cases where Patchscope performs worse than the baseline occur in later layers. Our interpretation is that given the language modeling training objective, the representations shift toward next-token prediction in the later layers. Therefore, the attribute of interest would not be as readily accessible via the model’s computation in these layers. This interpretation is also aligned with recent findings that show no decline in using linear relational embedding in predicting ω only when the next token also happens to be ω (Hernandez et al., 2023b). We postulate that when the immediate next token is not the attribute of interest, it does not necessarily mean the attribute information is lost, but rather it may not be as easily accessible on the surface. We hypothesize that using a Patchscope with a more expressive mapping f could improve attribute extraction accuracy in the later layers, which we leave for future work to verify.

Source-Target Layer Interplay Fig. 7 visualizes the interaction between ℓ and ℓ^* . These heatmaps show attribute extraction success rate for the zero-shot feature extraction Patchscope for a fixed (ℓ, ℓ^*) combination. The lower left quadrants show setups where both ℓ and ℓ^* represent early to middle layers, and the success rate is maximal. The right half of the heatmaps represents late ℓ , which achieves lower success rate due to token representations shifting toward next-token prediction as discussed earlier. In addition, we notice lower success rate in the top half of the heatmaps which represents late ℓ^* . It is worth noting that in this task, the accuracy is not only based on the immediate next-token prediction, but rather whether ω appears in the next 20 autoregressively generated tokens. The placeholder token “x” does still remain in the input, and its representation persists in the early layers in the target computation. This explains why a lower attribute extraction rate is observed in later ℓ^* values. We leave it to the future work to investigate adaptations to Patchscopes to control contamination from the placeholder tokens and make them more amenable to late ℓ^* choices.

Statistical Test Details The null hypothesis is: “There is no significant difference between probing accuracy and patchscope accuracy”. We calculate attribute extraction accuracy per layer, in a 40-layer model, across various tasks. Since we run a test for each task independently (see Tab. 2), we use the conservative Bonferroni correction to address the multiple comparison problem. That is, for a desired overall $\alpha = 0.05$, we consider $\alpha/\text{number of tasks} = \frac{0.05}{12} = 0.004$ for each individual task.

D. Additional Information and Results on the Entity Resolution Experiment

D.1. Experimental Setup

Recall that we use a few-shot target prompt template for decoding an entity description: “subject₁ : description₁, subject₂ : description₂, ..., subject_k : description_k, x”, while patching the last position which corresponds to “x”. Specifically, we use the following target prompt, obtained randomly: “Syria: Country in the Middle East, Leonardo DiCaprio: American actor, Samsung: South Korean multinational major appliance and consumer electronics corporation, x” and task the model to generate the completion after the patched representation in “x”. For the subject description, which is composed of $k = 3$ random subject entities, we used the wptools¹¹ python package for obtaining a description of every subject entity from Wikipedia.

D.2. Additional Quantitative Results

In this section, we present the Rouge1 (Lin, 2004) and SBERT score (Reimers & Gurevych, 2019) results, as well as the results for the Pythia models (Biderman et al., 2023).¹² In Fig. 8 and Fig. 9, we present the Rouge1 and the SBERT results, respectively, complementary to the RougeL results in Fig. 3 from §4.3. Note that for Pythia, the smaller model (6.9B) outperforms the larger one (12B), and hence, our cross-model patching method would not be able to improve the inspection of the smaller model, unlike the trends we observed for Vicuna.

D.3. Additional Qualitative Results

In this section, we provide more examples and discuss our observations about the gradual process of entity resolution. As shown in Tab. 6 and Tab. 7, it is interesting to observe that the resolution process for the same input can look different across models, suggesting they assign different likelihoods to different entities, and weigh context differently. For example,

¹¹<https://github.com/siznax/wptools/>

¹²We used the package from <https://www.sbert.net/>, with the sentence-transformers/all-MiniLM-L6-v2 model.